

Electricity Demand Forecasting using AI

at the Department of

Control Systems

Faculty IV – Electrical Engineering and Computer Science
Institute for Energy and Automation Technology

Authors:	Anton Valentin Dilg	(466011)
	Ramin Leon Neymeyer	(466437)
	Ramon Rennert	(466893)
Supervisor:	Dr. Daria Palis	
Submission date:	March 9, 2026	

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen, die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generische KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

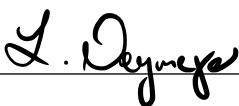
Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023 habe ich zur Kenntnis genommen.

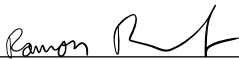
Berlin, den 9. März 2026



ANTON VALENTIN DILG



RAMIN LEON NEYMEYER



RAMON RENNERT

Contents

1. Introduction	1
2. Dataset	1
3. Forecasting Model	5
3.1. Backtesting	5
3.2. Seasonal Naïve Forecaster	6
3.3. Recursive Multi-Step Forecaster	7
3.4. Regression Estimators	8
3.5. Hyperparameter Tuning	9
3.6. Final Model	11
4. Discussion	12
4.1. Feature Importance	12
4.2. SHAP Values	15
4.3. Temporal Error	16
5. Conclusion and Outlook	17
References	18
A. Appendix	20
A.1. Complete Dataset	20
A.2. Tuned Hyperparameters	20

List of Figures

1.1.	50Hertz control area marked in red.	1
2.1.	Temporal patterns of electricity demand across multiple time scales. . . .	3
2.2.	Cyclic Encoding of the hour feature.	4
3.1.	Seasonal Naïve Forecaster with MAPE of 8.57%.	7
3.2.	Autocorrelation and partial autocorrelation of the grid load time series. .	7
3.3.	Recursive multi-step forecasting.	8
3.4.	Estimator comparison with exogenous features and different lags.	10
3.5.	Bayesian Optimization with 100 trials to efficiently explore the hyperpa- rameter space.	11
3.6.	Tuned XGBRegressor with a MAPE of 2.94%.	11
4.1.	Top 20 features ranked by gain and split count for LGBM and XGB. . .	14
4.2.	Sequential feature addition experiment added according to their impor- tance ranking (gain and split count) for LGBM and XGB.	14
4.3.	SHAP Summary of the most important features on the model output for LGBM and XGB.	15
4.4.	Temporal decomposition of MAPE by month, day of week and hour of day for the test year 2024.	16

List of Tables

3.1.	Temporal split of training and test data used for backtesting and evaluation.	6
4.1.	Forecasting performance of different models	12
A.1.	Overview of dataset features grouped by category.	20
A.2.	Tuned hyperparameters of the LGBMRegressor and XGBRegressor and lag structure are obtained via Bayesian optimization.	20

1. Introduction

This project focuses on the development of a Machine Learning (ML) model for short-term electricity demand forecasting. The analysis is restricted to the 50Hertz control area, which covers eastern Germany as well as the metropolitan region of Hamburg as depicted in Figure 1.1. The main objectives are the construction of a dataset including relevant exogenous variables and the prediction of hourly electricity demand with a forecasting horizon of up to 24 hours. Accurate short-term load forecasts are essential for efficient grid operation, market planning, and maintaining overall system stability.

All source code and datasets are publicly available in our GitHub repository under the MIT license¹.

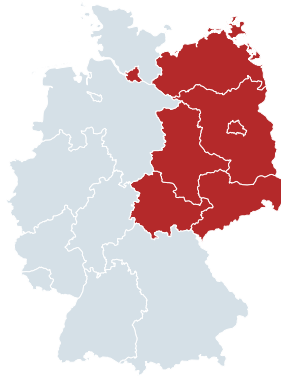


Figure 1.1: 50Hertz control area marked in red.

2. Dataset

The dataset combines electricity demand in the 50Hertz control area with power generation forecast by source, market prices, holiday indicators, and weather data from multiple meteorological stations.

Since the influence of exogenous variables on electricity demand is unknown a priori, all available features are retained during data preparation. This includes generation variables and a broad set of meteorological observations. Retaining the full feature set allows for exploratory analysis and model-based feature selection in later stages.

¹github.com/lar-tech/electricity-demand-forecasting

During data inspection, duplicate and missing hourly timestamps were observed due to daylight saving time (DST) transitions, during which one hour is repeated in autumn and omitted in spring. To ensure a unique and continuous time index required for forecasting, all timestamps are converted from local time to Coordinated Universal Time (UTC). Furthermore, hourly resampling is performed using left-closed, right-labeled intervals to prevent data leakage, ensuring that each aggregated value depends exclusively on past observations and is therefore suitable for forecasting applications.

Energy Demand Electricity demand and generation data for the 50Hertz control area from 2015 to 2024 are provided by the German Federal Network Agency (Bundesnetzagentur) via the SMARD API [1]. The primary forecasting target is the grid load, representing total electricity demand in the region.

To gain an initial understanding of the data, the time series are visualized. The annual profiles exhibit a high degree of similarity, indicating pronounced periodic behavior. To analyze characteristic intra-annual variations, the demand is averaged across all years and shown in Figure 2.1b. The resulting profile reveals a strong and stable seasonal pattern, with higher demand in winter and lower demand in summer, driven by increased heating, lighting, and indoor activity during colder months. This structure remains consistent throughout the observation period, suggesting no major structural changes in demand behavior between 2015 and 2024.

In addition to the annual cycle, a clear reduction in demand is observed around the Christmas and New Year period. This temporary decrease aligns with reduced industrial and commercial activity during public holidays and underlines the importance of calendar-related features in electricity demand forecasting. Furthermore, in Figure 2.1c it is clear that during the weekend the demand decreases for the same reason. The project's objective is a prediction of a complete day. Hence, the average grid load over a day is useful to see beforehand. Figure 2.1d illustrates a clear demand pattern, with low consumption at night and two peaks around 9am and 5pm. The morning peak results from the concurrent start of industrial and residential activity, while the evening peak reflects continued industrial demand combined with increased household consumption. As activity declines overnight, electricity demand reaches its daily minimum. This pattern is consistent across all seasons.

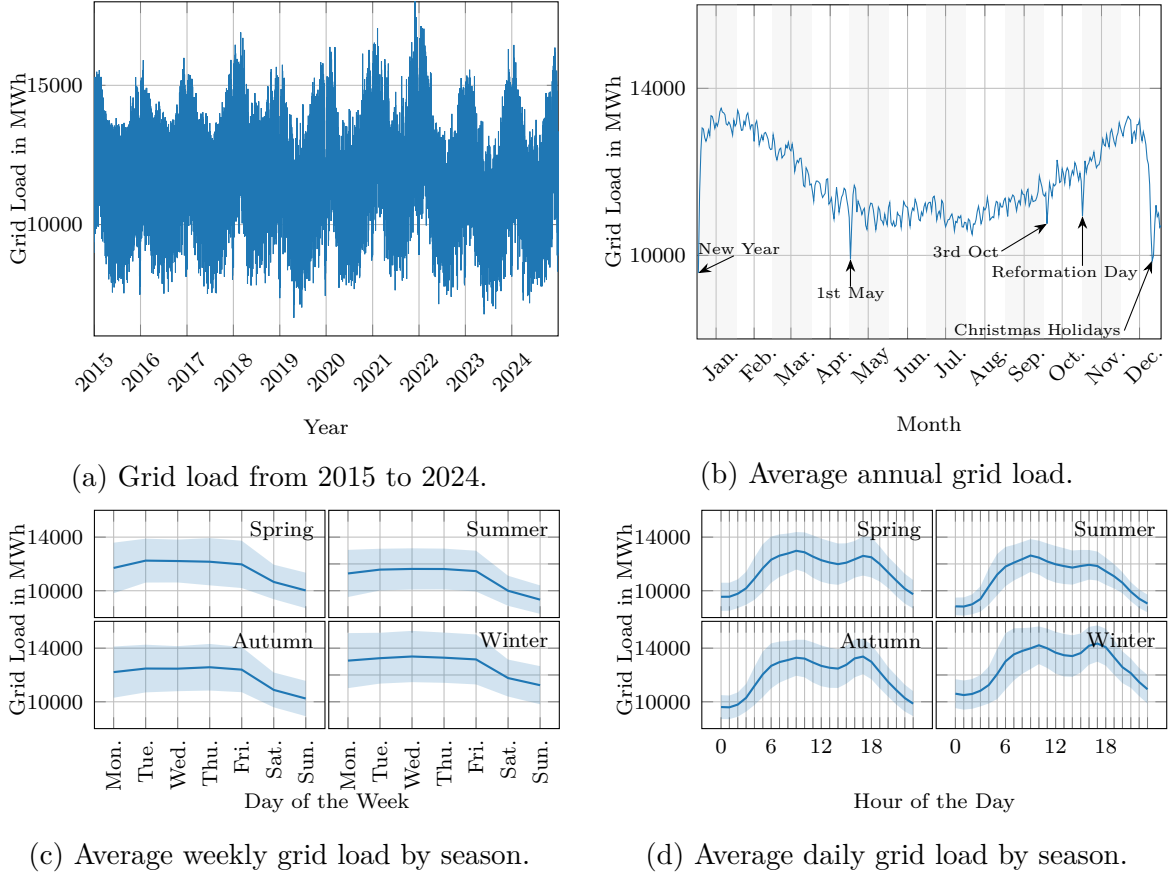


Figure 2.1: Temporal patterns of electricity demand across multiple time scales.

Temporal Features Temporal features are included to capture recurring demand patterns driven by human behavior and institutional schedules. Electricity consumption exhibits pronounced daily, weekly, and seasonal cycles that are not fully explained by physical variables alone. Accordingly, the dataset contains features representing the hour of the day, day of the week, week of the year and month.

Several temporal features (e.g., hour of day, day of week) are inherently cyclical. To preserve their periodic structure and avoid artificial discontinuities at time boundaries, these variables are encoded using sine and cosine transformations of the corresponding period. This trigonometric encoding efficiently captures temporal proximity and enables the model to learn smooth recurring patterns [2], [3]. The resulting encoding is illustrated in Figure 2.2.

Calendar effects are represented through public and school holidays as well as bridge days. Since public holidays vary across German federal states, a single feature is constructed by aggregating the number of states observing a holiday on each date. School

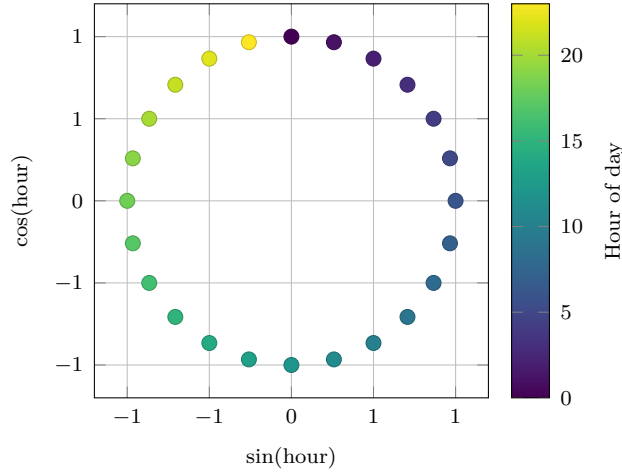


Figure 2.2: Cyclic Encoding of the hour feature.

holidays, which differ in timing and duration across regions, are included to account for systematic demand changes associated with reduced industrial activity and altered household behavior [4].

In addition, statistical rolling features such as the mean and standard deviation over the preceding seven days are included, allowing the model to access summary statistics of recent demand behavior without learning them implicitly from raw lag values. The dataset further contains the grid load from the equivalent weekday one year prior, where the equivalent date refers not to a fixed 365-day offset but to the nearest matching weekday, ensuring comparability in weekly demand structure across years.

Weather Data Weather variables are included to capture the impact of meteorological conditions on electricity demand and renewable generation. The data is obtained via the Meteostat Python library [5] and compiled from multiple stations distributed across the 50Hertz control area to ensure adequate spatial coverage. The selected stations represent the north-eastern, north-western, central, south-eastern, and south-western parts of the region. For each timestamp weather observations from the nearest station are assigned, allowing regional variation in temperature, wind and solar conditions to be reflected in the dataset. While the data consists of realized weather observations, it is treated as a proxy for weather forecasts, as actual forecast data was not available.

Market Data Market price data is obtained from the Energy-Charts API [6] and covers the DE/LU price zone. Until October 2018, Germany, Austria, and Luxembourg formed

a joint bidding zone (DE/AT/LU), after which Austria became a separate market area. Consequently, two price series are available in the dataset. For consistency over the full observation period and since the focus is on electricity demand in Germany, these series are combined into a single day-ahead market price variable.

The complete dataset can be found in the Appendix A.1.

3. Forecasting Model

Time series forecasting aims to predict future values of a target variable using historical observations and exogenous information. In contrast to standard regression, time series data is temporally ordered and exhibits serial dependencies, which violate the assumption of independent and identically distributed samples [7]. In the following section, the evaluation procedure is described, as conventional cross-validation is not applicable to causal time series. Furthermore, different forecasting strategies and regression estimators are analyzed, and the impact of exogenous variables on model accuracy is investigated.

3.1. Backtesting

To evaluate forecasting performance under realistic operational conditions, a backtesting framework is applied. Backtesting simulates the deployment of a forecasting model by repeatedly training it on historical data and generating forecasts for future time horizons [7]. In this project, a rolling-origin evaluation strategy is used. At each iteration, the model is trained on all data available up to a given point in time and then used to predict electricity demand for the subsequent 24 hours at an hourly resolution. The training window is incrementally extended, and the procedure is repeated over the full evaluation period from 2015 to 2023. Forecast performance is subsequently evaluated on an unseen test period from January 1, 2024 to December 31, 2024. An additional option is to refit the model at each forecasting step to better reflect practical operation, where models are regularly updated as new data becomes available. This approach is applied only to the final model, as using it in earlier experiments would unnecessarily increase computational cost.

Forecast accuracy is assessed using multiple error metrics. The Mean Squared Er-

ror (MSE) penalizes large deviations more strongly, thereby highlighting unstable predictions. It is commonly used during tuning and fitting because it remains stable throughout both processes. Additionally, the Mean Absolute Percentage Error (MAPE) provides a relative measure that facilitates comparison across different demand levels. The MSE is defined as

$$\text{MSE} = \frac{1}{N} \sum_{t=1}^N (y_t - \hat{y}_t)^2, \quad (3.1)$$

while the MAPE is given by

$$\text{MAPE} = \frac{1}{N} \sum_{t=1}^N \left| \frac{y_t - \hat{y}_t}{y_t} \right|. \quad (3.2)$$

The temporal split between training and testing data is summarized in Table 3.1. The training period spans multiple years to capture long-term seasonal patterns and structural variability, while the test period comprises a full calendar year to enable a comprehensive and unbiased evaluation under unseen conditions.

Table 3.1: Temporal split of training and test data used for backtesting and evaluation.

Dataset	Start date	End date	Duration
Training period	2015-01-01	2023-12-31	9 years
Test period	2024-01-01	2024-12-31	1 year

3.2. Seasonal Naïve Forecaster

A simple baseline model is introduced to establish a reference level of forecasting performance. Baseline models are used to assess whether more complex approaches provide meaningful improvements over simple heuristics [8].

As a baseline, a seasonal naïve forecasting method is employed, which predicts future electricity demand by reusing observations from the same hour in previous periods. This approach exploits recurring seasonal patterns and is commonly used as a benchmark in short-term electricity demand forecasting [8]. The forecasting principle is illustrated in Figure 3.1. Using this baseline model, an average forecasting error of 8.57% is obtained, measured over the entire year of 2024. This result serves as a lower performance bound that all advanced models are expected to outperform.

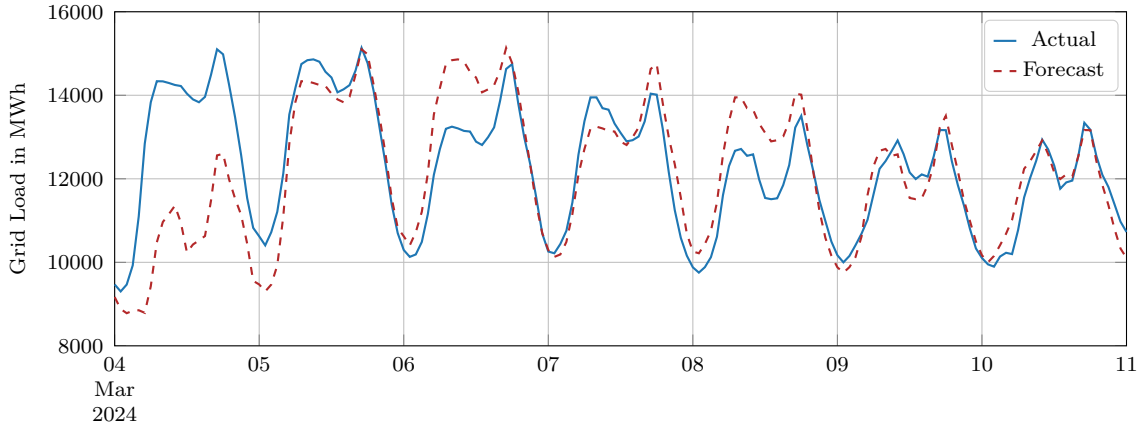


Figure 3.1: Seasonal Naïve Forecaster with MAPE of 8.57%.

3.3. Recursive Multi-Step Forecaster

To achieve the objective of forecasting multiple values at once, there are several strategies to handle this kind of problem. Two commonly used approaches are the so-called recursive and direct forecasters. Both strategies come with their own advantages and trade-offs. The direct approach can yield more accurate forecasts for longer horizons, but it is computationally expensive, since a separate model has to be trained for each forecast step. To decide whether a recursive forecasting strategy is appropriate for this problem, it is helpful to analyze the autocorrelation structure of the time series.

The Autocorrelation Function (ACF) in Figure 3.2a exhibits a clear periodic pattern, indicating strong temporal dependencies across multiple time lags. Pronounced peaks at multiples of 24 hours highlight daily seasonality, suggesting that past demand values from previous days contain substantial information about future demand. The Partial

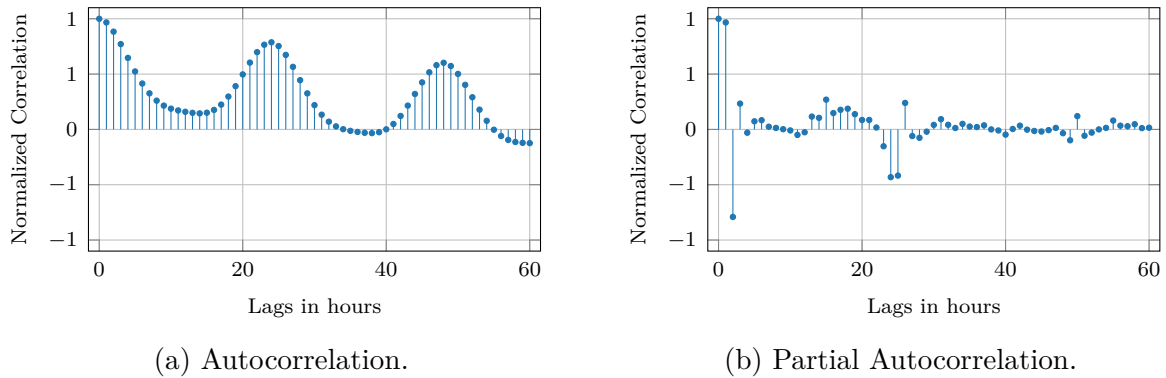


Figure 3.2: Autocorrelation and partial autocorrelation of the grid load time series.

Autocorrelation Function (PACF) in Figure 3.2b further shows that only a limited number of lower-order lags have a significant direct influence, while higher-order correlations are largely explained indirectly through these lags. Together, these observations indicate a strong autoregressive structure in the data and support the use of a recursive forecasting approach for the considered prediction horizon.

The recursive approach uses a single model and feeds previous predictions back into the model to generate the full forecast horizon. In Figure 3.3 an example is visualized. The model sees the blue circled values and predicts the new value. The blue circled values are the lags and indicate how much the model is allowed to look back into the past. Next it uses the prediction and the other two lags before to predict the next value like a moving window.

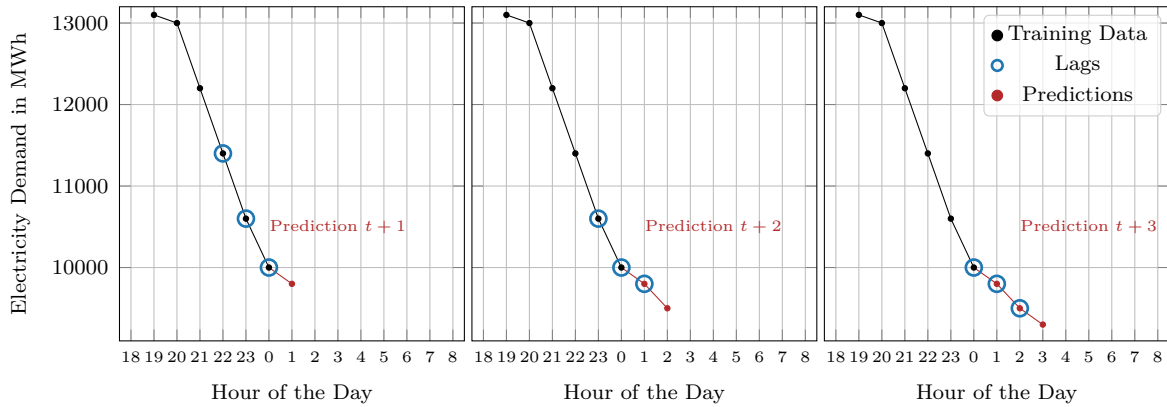


Figure 3.3: Recursive multi-step forecasting.

3.4. Regression Estimators

The forecaster defines which lagged values and exogenous features are provided as inputs to the predictive model. The actual component responsible for estimating future values of the time series is the regression estimator. Various types of estimators can be employed, including linear models and tree-based methods, each associated with specific advantages and limitations.

As a representative linear approach, Ridge regression is considered. This model captures linear relationships between lagged electricity demand and future demand values. However, Ridge regression requires a complete feature matrix without missing values. This assumption cannot always be satisfied when incorporating exogenous variables such as weather data or market prices. Consequently, Ridge regression is not considered further.

Tree-based gradient boosting methods are therefore examined, specifically Light Gradient Boosting Machine (LGBM) and eXtreme Gradient Boosting (XGB). Both are based on the gradient boosting framework [9], which constructs an additive model by sequentially fitting weak learners to the negative gradient of the loss function. Starting from an initial estimate $F_1(x) = \bar{y}$, each subsequent learner f_{m+1} is trained on the pseudo-residuals

$$\hat{r}_{mi} = - \left. \frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_m}, \quad i \in \{1, \dots, N\}, \quad (3.3)$$

and the model is updated as $F_{m+1}(x) = F_m(x) + \gamma_{m+1}f_{m+1}(x)$, where the step size γ_{m+1} is chosen to minimize the total loss. Compared to ensemble methods such as random forests, gradient boosting estimators offer substantially faster training times while retaining the ability to model complex, nonlinear relationships. LGBM employs histogram-based splitting in combination with a leaf-wise tree growth strategy, resulting in high computational efficiency [10]. In contrast, XGB applies stronger regularization and a level-wise tree growth strategy, which leads to more stable model behavior at the cost of increased computational effort [11].

Another relevant aspect is the selection of the number of lagged values used as model inputs, as well as the contribution of exogenous features to forecasting performance. As illustrated in Figure 3.4, models that incorporate exogenous variables consistently achieve lower MAPE values than their counterparts relying solely on lagged demand values, across all evaluated lag configurations. This observation suggests that exogenous information provides additional predictive value for the forecasting task. In contrast, the influence of the number of lags is less conclusive. Increasing the number of lagged inputs does not lead to a systematic improvement in performance and no monotonic relationship between lag count and forecast accuracy can be observed. These results indicate that lag selection represents a non-trivial hyperparameter choice and should be optimized carefully rather than increased arbitrarily.

3.5. Hyperparameter Tuning

After selecting the estimator and the overall model structure, the next step is to fine-tune the model's hyperparameters. Up to this point, all models have been evaluated using default parameter settings. Since the number of hyperparameters and possible lag configurations is large, manual tuning or exhaustive evaluation would be highly time-

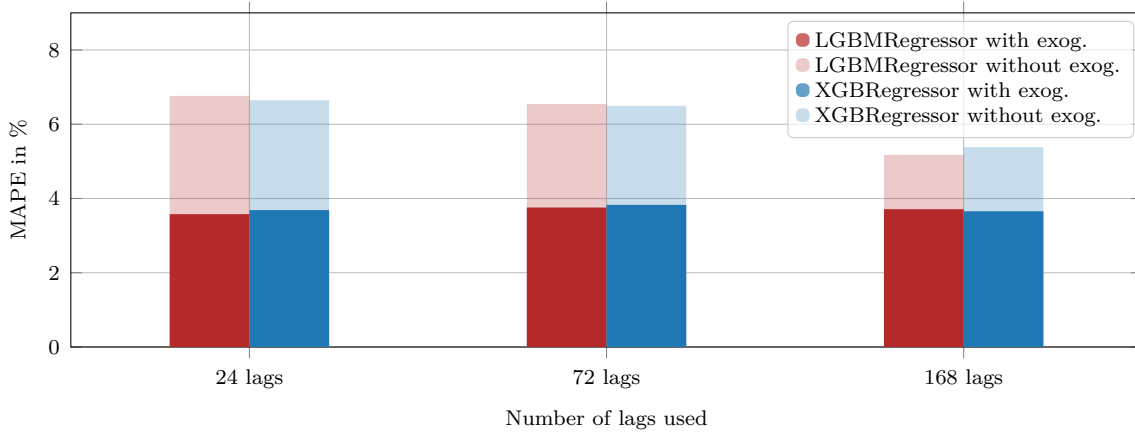


Figure 3.4: Estimator comparison with exogenous features and different lags.

consuming and computationally inefficient.

Two common approaches for hyperparameter tuning are grid search and Bayesian optimization. Grid search evaluates all combinations of a predefined parameter grid, which becomes infeasible in high-dimensional search spaces. Therefore, Bayesian hyperparameter optimization is employed, as it enables a more efficient exploration of the parameter space by incorporating information from previous evaluations.

Formally, Bayesian optimization aims to minimize an objective function

$$\theta^* = \arg \min_{\theta \in \Theta} f(\theta), \quad (3.4)$$

where $f(\theta)$ denotes the validation error of the model with hyperparameters θ . A probabilistic surrogate model, typically a Gaussian process, is used to approximate $f(\theta)$ [12]. Based on this surrogate, an acquisition function $\alpha(\theta)$ determines the next hyperparameter configuration to evaluate by balancing exploration and exploitation.

For the optimization objective, the MSE is used instead of the MAPE. The MAPE, as defined in Equation 3.2, can become unstable when the true values y_t are close to zero, as small absolute errors may lead to disproportionately large percentage errors. In contrast, the MSE, as described in Equation 3.1, provides a smoother and more stable optimization objective, which is particularly important for Bayesian optimization, as the surrogate model relies on consistent error evaluations.

The Bayesian optimization was performed using 100 trials for each estimator, requiring approximately 80 minutes of computation time in total. As illustrated in Figure 3.5, the

tuning process results in a slight but consistent reduction of the validation error for both models over successive trials. In terms of final forecasting performance, the tuned XGB model reduces the MAPE from 3.66% to 3.15%, while the tuned LGBM model achieves a reduction from 3.71% to 3.16%. Although the absolute improvements are moderate, they are consistent across trials and justify the additional computational effort associated with hyperparameter tuning. A complete list of the tuned hyperparameters as well as optimal lags is provided in Appendix A.2.

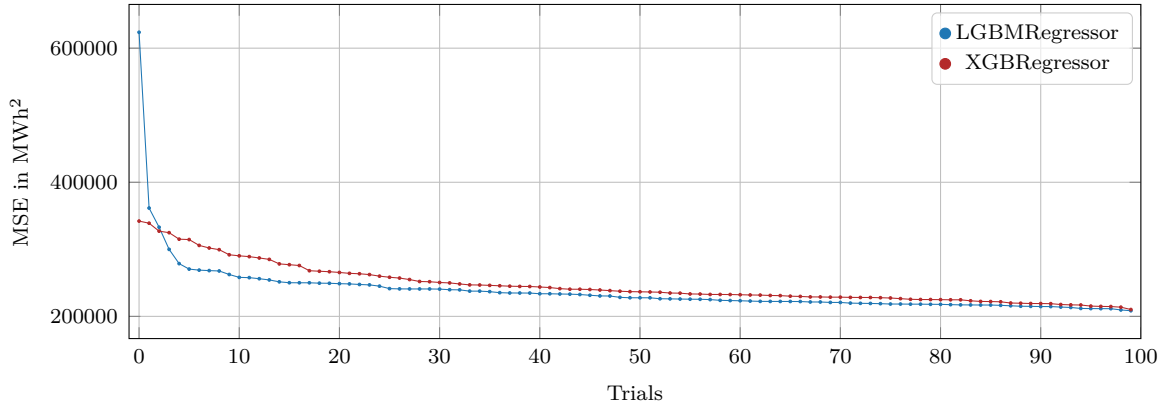


Figure 3.5: Bayesian Optimization with 100 trials to efficiently explore the hyperparameter space.

3.6. Final Model

The best-performing model achieves a MAPE of 2.94%. This result is obtained using an iterative refitting strategy, in which the model is retrained at each forecasting step using

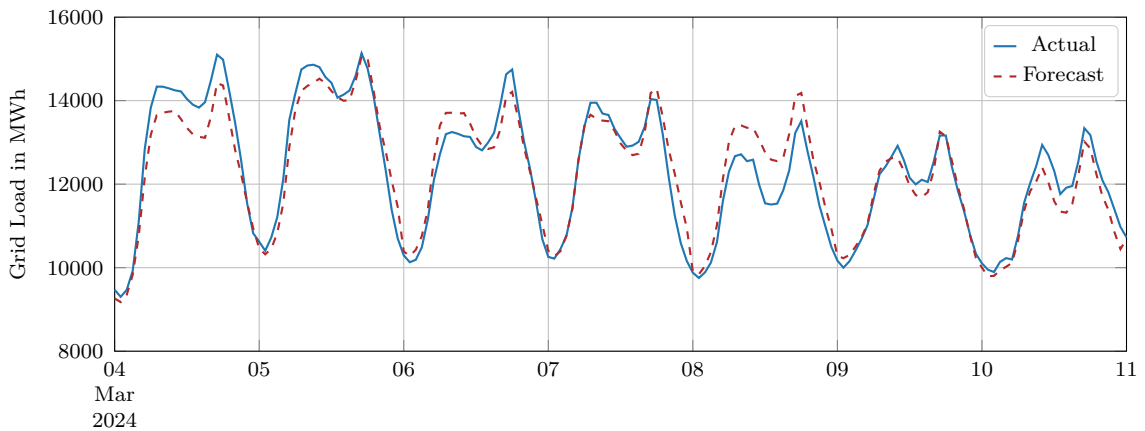


Figure 3.6: Tuned XGBRegressor with a MAPE of 2.94%.

all newly available observations. As a consequence, newly observed data are continuously incorporated into the training set, allowing the model to adapt to evolving demand patterns over time. The final forecasting setup consists of a tuned XGB estimator embedded in an autoregressive forecasting framework and augmented with all available exogenous features. The resulting performance is illustrated in Figure 3.6. Refitting and evaluating the model over a one-year period requires approximately 40 minutes of computation time when using two CPU cores, representing a reasonable trade-off between predictive accuracy and computational effort.

4. Discussion

In the following section, the training results and the performance of the final forecasting model are discussed and interpreted.

Table 4.1 summarizes the forecasting performance of all evaluated models. A clear performance gap can be observed between models relying solely on lagged demand values and those incorporating exogenous features.

Table 4.1: Forecasting performance (MAPE in %) of different models with and without exogenous features.

Model	Without exog. features	With exog. features
Seasonal Naive	8.57	–
LGBM (default)	6.76	3.71
XGB (default)	6.64	3.66
LGBM (tuned)	–	3.16
XGB (tuned)	–	3.15
LGBM (tuned and refitted)	–	3.01
XGB (tuned and refitted)	–	2.94

4.1. Feature Importance

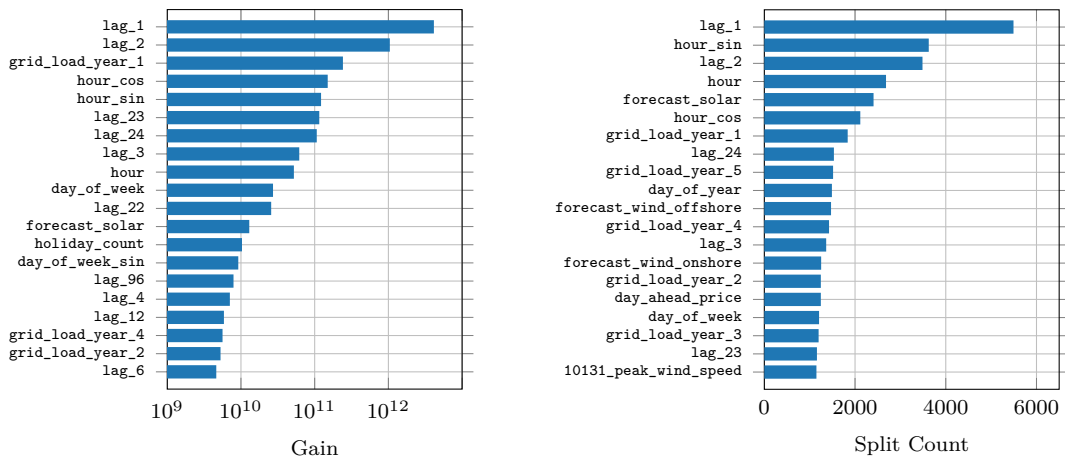
Figure 4.1 shows the top 20 features ranked by gain and split count for both estimators. In terms of gain, `lag_1` and `lag_2` dominate by a large margin in both models, confirming the strong autoregressive structure identified through the ACF and PACF analysis in Section 3.3. The third most important feature by gain is `grid_load_year_1`, indicating that year-over-year demand patterns provide substantial predictive value. Temporal

features such as `hour_cos`, `hour_sin`, and `day_of_week` also contribute meaningfully, reflecting the pronounced daily and weekly demand cycles observed in the exploratory analysis.

The split count ranking reveals a complementary perspective. While `lag_1` remains the most frequently used feature, temporal variables such as `hour_sin`, `hour`, and `forecast_solar` are split on far more often relative to their gain contribution. This suggests that these features serve as fine-grained routing variables, enabling the trees to partition the input space into distinct demand regimes, even though individual splits yield comparatively small reductions in the loss function. Notably, `forecast_solar` ranks among the top five by split count in both models but appears lower in the gain ranking, indicating that solar generation forecasts contribute through many small, context-dependent adjustments rather than a few high-impact splits.

Both estimators produce broadly consistent importance rankings, with minor differences in ordering. For instance, XGB assigns higher gain to `lag_24` compared to LGBM, while LGBM ranks `hour_cos` higher. These differences likely reflect the distinct tree growth strategies of the two algorithms, leaf-wise for LGBM and level-wise for XGB, rather than fundamental disagreements about feature relevance.

To quantify the marginal contribution of individual features, a sequential feature addition experiment is conducted. Starting from the most important feature, additional features are added one at a time according to their importance ranking, and the resulting MAPE is evaluated at each step. The results are shown in Figure 4.2. Across all four configurations, the MAPE decreases sharply within the first five to ten features and



(a) LGBM.

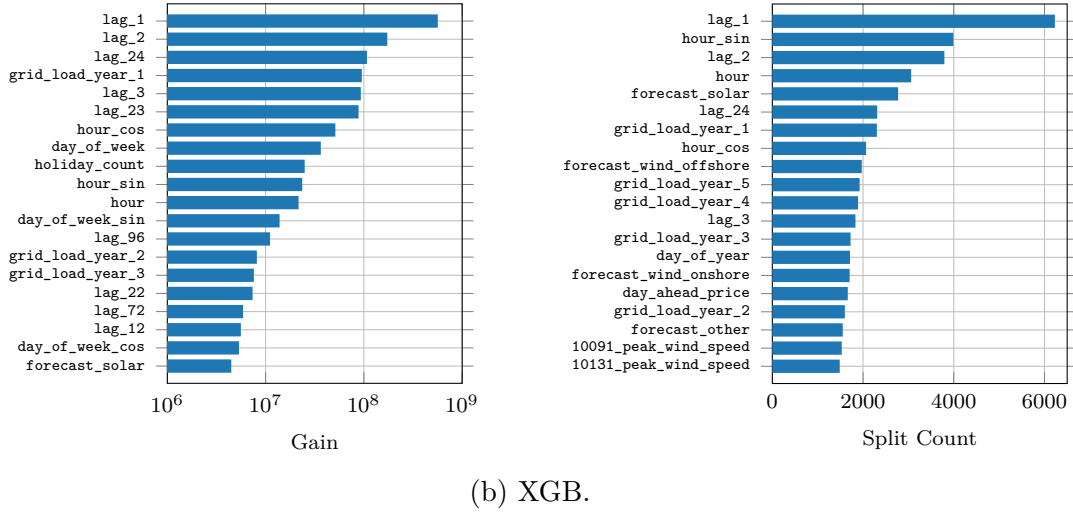


Figure 4.1: Top 20 features ranked by gain and split count for LGBM and XGB.

converges beyond approximately 20 to 30 features.

This indicates that the majority of predictive information is captured by a relatively small subset of features, while the remaining variables, primarily individual weather station measurements and higher-order lags, contribute only marginally.

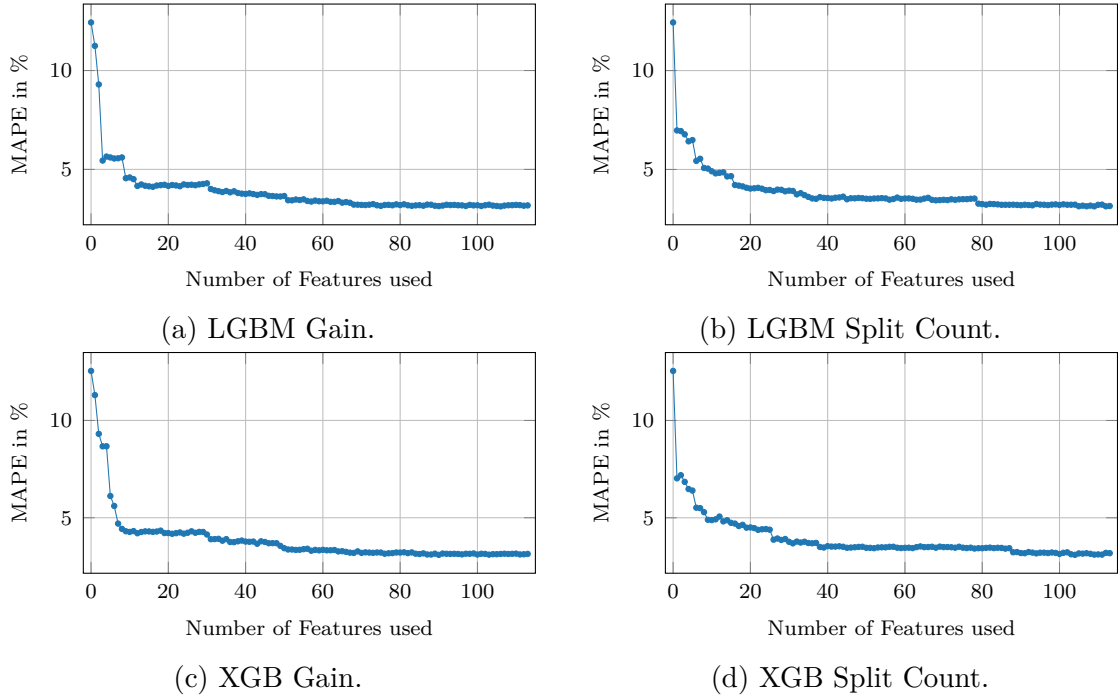


Figure 4.2: Sequential feature addition experiment added according to their importance ranking (gain and split count) for LGBM and XGB.

The gain-based ranking generally leads to a faster initial decrease in MAPE compared to the split count ranking, suggesting that gain provides a more effective ordering for greedy feature selection in this application.

4.2. SHAP Values

While feature importance quantifies how much each variable contributes to the model overall, it does not reveal the direction of its effect. SHapley Additive exPlanations (SHAP) values address this by assigning each feature a signed contribution to every individual prediction, grounded in cooperative game theory [13]. Figure 4.3 shows the SHAP summary plots for both estimators.

The feature `lag_1` exhibits a clear monotonic relationship: high feature values, i.e., high recent demand, push predictions upward, while low values push them downward. The wide spread of SHAP values, ranging from approximately -4000 to $+4000$ MWh, confirms that the model anchors heavily on the most recent observation. This is consistent with the dominant role of `lag_1` in the feature importance analysis and reflects the strong short-term autocorrelation of electricity demand.

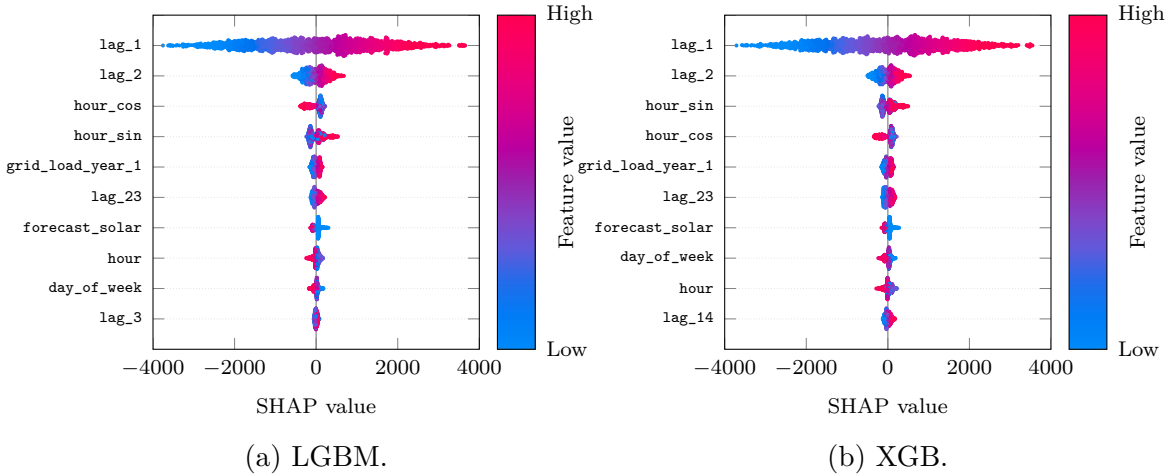


Figure 4.3: SHAP Summary of the most important features on the model output for LGBM and XGB.

The remaining top features follow interpretable patterns consistent with the demand structure discussed in Section 2. The cyclically encoded features `hour_cos` and `hour_sin` display clustered SHAP values corresponding to distinct demand regimes throughout the day, while `grid_load_year_1` and `lag_23` show positive SHAP values for high historical

demand, reinforcing the seasonal and daily periodicity. More generally, all lag and temporal features exhibit a monotonic relationship, higher past or structurally similar demand increases the prediction, whereas `forecast_solar` acts in the opposite direction. This is physically plausible, as increased solar generation displaces conventional load from the grid, effectively lowering the net demand observed in the grid load signal. Both SHAP plots are highly consistent across LGBM and XGB, confirming that both estimators have learned similar representations of the underlying demand drivers.

4.3. Temporal Error

Figure 4.4 decomposes the forecasting error across three temporal dimensions. The monthly breakdown shows that both ML models maintain a relatively stable MAPE between 2.5% and 3.5% throughout the year, while the Seasonal Naïve Forecast fluctuates more strongly. December exhibits the highest error across all models due to irregular demand patterns during the Christmas holiday period. The weekly breakdown reveals that Sundays are most difficult to forecast, whereas midweek days yield the lowest errors. This is consistent with the more regular and industry-driven demand patterns observed on working days.

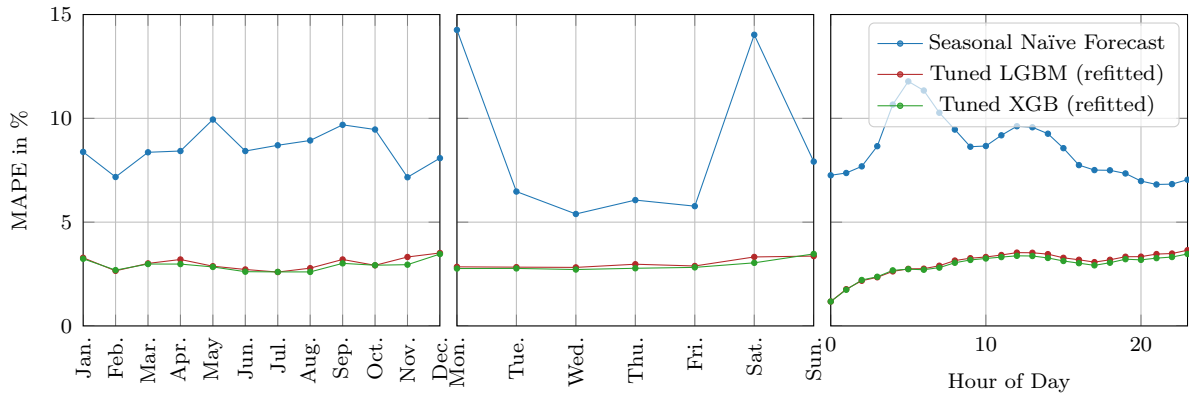


Figure 4.4: Temporal decomposition of MAPE by month, day of week and hour of day for the test year 2024.

The hourly profile shows the most pronounced variation. Error is lowest during the nighttime hours and increases throughout the day, peaking in the late afternoon and evening. Since the backtesting framework generates forecasts starting at midnight, this pattern reflects two confounded effects: the inherent difficulty of predicting demand

during transition periods with steep ramps and the accumulation of recursive prediction errors over the 24-hour forecast horizon. Notably, the Seasonal Naïve Forecast exhibits its largest errors during the morning ramp-up between 4:00 and 8:00, which is precisely the period where the ML models achieve the greatest relative improvement.

5. Conclusion and Outlook

This project developed a ML pipeline for short-term electricity demand forecasting in the 50Hertz control area. Starting from a Seasonal Naïve baseline with a MAPE of 8.57%, the final model, a tuned and refitted XGB estimator embedded in a recursive forecasting framework, achieved a MAPE of 2.94% on the 2024 test year. The inclusion of exogenous features proved to be the single most impactful factor, roughly halving the forecasting error compared to models relying solely on lagged demand values. Feature importance and SHAP analysis confirmed that the model relies primarily on recent demand observations, cyclical temporal encodings, and year-over-year load patterns, while solar generation forecasts contribute a physically interpretable inverse effect. The temporal error analysis revealed that forecasting accuracy is highest during nighttime and midweek periods and degrades during holiday periods, weekends and later hours of the forecast horizon. Several limitations should be noted. The weather data used in this study consists of realized observations rather than actual meteorological forecasts, meaning that the reported accuracy represents an optimistic estimate of operational performance. Furthermore, the backtesting framework generates forecasts exclusively from midnight, which prevents a clean separation between hour-of-day effects and recursive error accumulation. The analysis is also restricted to a single control area, and the models produce only deterministic point forecasts without quantifying prediction uncertainty. Future work could address these limitations in several directions. Incorporating actual weather forecasts instead of realized observations would provide a more realistic assessment of operational accuracy. Extending the analysis to additional control areas would test the generalizability of the proposed approach. Probabilistic forecasting methods, such as conformal prediction intervals [14] or quantile regression [15], could complement the point forecasts with uncertainty estimates that are valuable for risk-aware grid operation. Finally, a comparison with deep learning architectures such as Temporal Fusion Transformers could reveal whether the remaining forecasting error can be further reduced by capturing more complex temporal dependencies [16].

References

- [1] Bundesnetzagentur, *SMARD – Strommarktdaten für Deutschland*, smard.de, (Accessed: Nov. 13, 2025), 2025.
 - [2] J. Amat Rodrigo and J. Escobar Ortiz, *skforecast*, version 0.19.1, Dec. 2025. DOI: 10.5281/zenodo.8382788
 - [3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., *Scikit-learn: Machine learning in Python*, *Journal of Machine Learning Research*, vol. 12, no. Oct, pp. 2825–2830, Oct. 2011. DOI: 10.5555/1953048.2078195
 - [4] M. N. Johannsen and C. Sanders, *Schulferien, Feiertage und Kalender in Deutschland*, schulferien.org, (Accessed: December 01, 2025), 2001.
 - [5] C. S. Lamprecht, *Meteostat Python*, meteostat.net, (Accessed: November 12, 2025), 2020.
 - [6] B. Burger, *Energy-Charts API*, energy-charts.info, (Accessed: 2026-01-16), 2025.
 - [7] M. Fotakis, *The Validity of Backtesting for Evaluation of Autoregressive Time Series Predictions*, M.S. thesis, Tilburg University, May 2021.
 - [8] N. Beck, J. Dovern, and S. Vogl, *Mind the Naive Forecast! A Rigorous Evaluation of Forecasting Models for Time Series with Low Predictability*, *Applied Intelligence*, vol. 55, p. 395, Feb. 2025. DOI: 10.1007/s10489-025-06268
 - [9] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. New York, NY, USA: Springer New York Inc., 2001. DOI: 10.1007/b94608
 - [10] Y. Shi, G. Ke, D. Soukhavong, J. Lamb, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, et al., *lightgbm: Light Gradient Boosting Machine*, 2025.
 - [11] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785
 - [12] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. Cambridge, Massachusetts, USA: The MIT Press, 2006. DOI: 10.7551/mitpress/3206.001.0001
-

-
- [13] S. M. Lundberg and S.-I. Lee, *A unified approach to interpreting model predictions*, in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Red Hook, NY, USA: Curran Associates Inc., 2017, pp. 4768–4777. DOI: 10.5555/3295222.3295230
 - [14] M. Zaffran, O. Féron, Y. Goude, J. Josse, and A. Dieuleveut, *Adaptive conformal predictions for time series*, in *International Conference on Machine Learning*, Baltimore, Maryland USA: PMLR, Jul. 2022, pp. 25 834–25 866.
 - [15] T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli, and R. J. Hyndman, *Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond*, *International Journal of Forecasting*, vol. 32, no. 3, pp. 896–913, Mar. 2016. DOI: <https://doi.org/10.1016/j.ijforecast.2016.02.001>
 - [16] B. Lim, S. Ö. Arik, N. Loeff, and T. Pfister, *Temporal Fusion Transformers for interpretable multi-horizon time series forecasting*, *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, Jun. 2021. DOI: <https://doi.org/10.1016/j.ijforecast.2021.03.012>
-

A. Appendix

A.1. Complete Dataset

Table A.1 provides a high-level overview of the dataset structure, summarizing the different feature groups without listing individual weather stations or lag variables explicitly.

Table A.1: Overview of dataset features grouped by category.

Category	Features
Target variable	grid_load
Calendar features	holiday_count, school_holiday
Temporal indices	hour, day_of_week, week, month, day_of_year, roll_mean_72, roll_std_168, roll_min_168, roll_max_168
Market data	day_ahead_price
Weather data	{station_code}_temperature, {station_code}_dew_point, {station_code}_relative_humidity, {station_code}_precipitation, {station_code}_snow_depth, {station_code}_wind_direction, {station_code}_average_wind_speed, {station_code}_peak_wind_speed, {station_code}_average_sea_level_air_pressure, {station_code}_sunshine_duration, {station_code}_weather_condition_code
Generation forecasts	forecast_wind_offshore, forecast_wind_onshore, forecast_solar, forecast_other
Cyclical encodings	hour_sin, hour_cos, day_of_week_sin, day_of_week_cos, week_sin, week_cos, month_sin, month_cos
Lag features	grid_load_year_1 – grid_load_year_8

A.2. Tuned Hyperparameters

Table A.2 summarizes the final hyperparameter configurations and selected lag structures for both gradient boosting models.

Table A.2: Tuned hyperparameters of the LGBMRegressor and XGBRegressor and lag structure are obtained via Bayesian optimization.

Hyperparameter	LGBMRegressor	XGBRegressor
learning_rate	0.02415	0.02134
n_estimators	2100	2800
max_depth	6	6
subsample	0.735	0.771
colsample_bytree	0.619	0.625
reg_alpha	8.40×10^{-5}	1.49×10^{-6}
reg_lambda	5.33×10^{-5}	2.46×10^{-8}
num_leaves	86	–
min_child_samples	28	–
min_child_weight	–	27.17
gamma	–	8.96
Lags	{1, 2, ..., 24, 48, 72, 96}	
